

COMPTE RENDU

Programmation Parallèle — TP1 — Hands-on MPI – Hello World,
Pair/Impair, Ping-Pong, Anneau, Maître/Esclave, Somme Parallèle
4e année Cybersécurité - École Supérieure d'Informatique et du
Numérique (ESIN)
Collège d'Ingénierie & d'Architecture (CIA)

Étudiant :	HATHOUTI Mohammed Taha KABORE Mohammad Sharif Jonathan
Filière :	Cybersécurité
Année :	2026/2027
Enseignant :	M. BAKHOUYA
Date :	26 septembre 2026

Table des matières

Rappel des objectifs du TP	2
Architecture du projet	2
Exercice Hello World	3
0.1 Principe	3
0.2 Résultat	3
1 Exercice 1 : Pair/Impair	3
1.1 Principe	3
2 Exercice 2 : Ping-Pong	4
2.1 Programme 1 : ping simple	4
2.2 Programme 2 : ping-pong chronométré	4
2.3 Résultat	4
3 Exercice 3 : Anneau (Ring)	4
3.1 Principe	4
3.2 Résultat	5
4 Exercice 4 : Maître/Esclave	5
4.1 Principe	5
4.2 Résultat	6
5 Exercice 5 : Somme Parallèle	6
5.1 Principe et répartition de la charge	6
5.2 Complexité théorique	8
5.3 Protocole de mesure	8
5.4 Résultats expérimentaux	9
5.5 Temps d'exécution en fonction de la taille du tableau	10
5.6 Temps d'exécution en fonction du nombre de processus	11
5.7 Accélération en fonction du nombre de processus	12
5.8 Interprétation : la loi d'Amdahl à l'œuvre	12
Conclusion	13

Rappel des objectifs du TP

Ce TP a pour objectif de prendre en main la bibliothèque **MPI (Message Passing Interface)** à travers une série d'exercices progressifs illustrant les schémas de communication classiques du calcul parallèle par passage de messages :

- **Hello World** : Initialisation d'un programme MPI, identification du rang et de la taille du communicateur
- **Pair/Impair** : Branchement conditionnel selon le rang du processus
- **Ping-Pong** : Communication point-à-point (`MPI_Send/MPI_Recv`) entre deux processus, avec et sans mesure de temps
- **Anneau (Ring)** : Communication en topologie circulaire, chaque processus ne parlant qu'à ses deux voisins
- **Maître/Esclave** : Diffusion d'une donnée du processus maître vers tous les esclaves
- **Somme Parallèle** : Calcul distribué de la somme d'un grand tableau, avec **mesure de performance** (temps séquentiel vs parallèle, accélération) pour différentes tailles de tableau et différents nombres de processus

L'objectif principal du dernier exercice est de **mesurer et comparer les performances** de la somme parallèle en fonction de :

- La **taille du tableau** à sommer
- Le **nombre de processus MPI** utilisés

Les résultats expérimentaux seront confrontés à la loi d'Amdahl pour expliquer les limites observées de l'accélération.

Architecture du projet

Le projet est organisé en modules, chacun implémentant un exercice, assemblés dans `main.c` et exécutés séquentiellement au sein du même run `mpirun` (séparés par des `MPI_Barrier` pour garder une sortie lisible) :

- `Hello.c / Hello.h` — Exercice Hello World
- `PairImpair.c / PairImpair.h` — Exercice 1 : Pair/Impair
- `PingPong1.c, PingPong2.c` — Exercice 2 : Ping-Pong simple puis chronométré
- `Ring.c / Ring.h` — Exercice 3 : Anneau
- `MasterSlave.c / MasterSlave.h` — Exercice 4 : Maître/Esclave
- `SommeParallele.c / SommeParallele.h` — Exercice 5 : Somme parallèle et mesures de performance
- `commun.c / commun.h` — Fonctions utilitaires partagées
- `Makefile` — Compilation (`make`), exécution (`make run`), benchmark (`make bench`) et génération des graphes (`make graphs`)

Le benchmark de l'exercice 5 est lancé avec le nombre de cœurs disponibles sur la machine (`NPMAX = nproc`), et teste lui-même en interne tous les sous-comptes de processus `{1, 2, 4, 8, 16}` inférieurs ou égaux à `NPMAX`, en formant des sous-communicateurs avec `MPI_Comm_split`.

Exercice Hello World

0.1 Principe

Chaque processus initialise l'environnement MPI (`MPI_Init`), récupère son rang (`MPI_Comm_rank`) et la taille du communicateur (`MPI_Comm_size`), puis affiche un message d'identification incluant le nom de la machine hôte (`MPI_Get_processor_name`).

```
void afficher_hello(int rang, int taille) {
    int namelen;
    char processor_name[MPI_MAX_PROCESSOR_NAME];

    MPI_Get_processor_name(processor_name, &namelen);

    printf("Hello, world, I am %d of %d running on the\n",
           machine %s\n",
           rang, taille, processor_name);
}
```

0.2 Résultat

Avec 16 processus lancés en `--oversubscribe` sur une machine à moins de 16 cœurs, chaque processus affiche correctement son rang unique de 0 à 15 et le nom de la machine, confirmant que le communicateur `MPI_COMM_WORLD` est bien formé.

1 Exercice 1 : Pair/Impair

1.1 Principe

Chaque processus teste la parité de son propre rang (`rang % 2`) et affiche un message différent selon qu'il est pair ou impair. Cet exercice illustre la **divergence de contrôle** entre processus SPMD (Single Program, Multiple Data) : tous exécutent le même code binaire, mais empruntent des branches différentes selon leur rang.

```
int est_pair(int rang) {
    return (rang % 2) == 0;
}

void afficher_parite(int rang) {
    if (est_pair(rang)) {
        printf("Je suis le processus pair de rang %d\n",
               rang);
    } else {
        printf("Je suis le processus impair de rang %d\n",
               , rang);
    }
}
```

2 Exercice 2 : Ping-Pong

2.1 Programme 1 : ping simple

Le processus 0 génère un tableau de valeurs aléatoires et l'envoie au processus 1 avec `MPI_Send` ; le processus 1 le reçoit avec `MPI_Recv` et affiche la dernière valeur reçue. Cette communication est **unidirectionnelle** et illustre la forme la plus basique d'échange point-à-point.

2.2 Programme 2 : ping-pong chronométré

Le processus 0 envoie le tableau au processus 1, qui le lui renvoie immédiatement tel quel ; le temps total d'aller-retour est mesuré avec `MPI_Wtime()` autour de la paire `Send/Recv`.

```
void pingpong_emetteur(double valeurs[], int nb_valeurs, int
    etiquette) {
    MPI_Status statut;
    double temps_debut, temps_fin;

    temps_debut = MPI_Wtime();
    MPI_Send(valeurs, nb_valeurs, MPI_DOUBLE, 1, etiquette,
        MPI_COMM_WORLD);
    MPI_Recv(valeurs, nb_valeurs, MPI_DOUBLE, 1, etiquette,
        MPI_COMM_WORLD, &statut);
    temps_fin = MPI_Wtime();

    printf("... en %f secondes.\n", temps_fin - temps_debut);
}
```

2.3 Résultat

Sur 1000 valeurs de type `double`, l'aller-retour complet (send + recv sur les deux processus) est mesuré à 46 μs , ce qui donne un ordre de grandeur de la latence de communication MPI pour de petits volumes de données sur la machine de test.

3 Exercice 3 : Anneau (Ring)

3.1 Principe

Les processus sont organisés en anneau logique : chaque processus de rang r ne communique qu'avec son **suivant** $(r+1) \bmod n$ et son **précédent** $(r-1+n) \bmod n$. Le processus 0 initialise un message (un compteur entier) et l'envoie à son suivant. Le message circule ensuite dans l'anneau ; à chaque passage par le processus 0, le compteur est décrémenté. Le message continue de circuler jusqu'à atteindre 0, moment où chaque processus recevant cette valeur termine sa boucle et transmet une dernière fois avant de sortir.

```
void calculer_voisins(int rang, int taille, int *suivant, int *
    precedent) {
    *suivant = (rang + 1) % taille;
```

```

        *precedent = (rang + taille - 1) % taille;
    }

    void faire_circuler_message(int rang, int suivant, int precedent,
                               int etiquette) {
        int message;

        while (1) {
            MPI_Recv(&message, 1, MPI_INT, precedent,
                    etiquette, MPI_COMM_WORLD, MPI_STATUS_IGNORE);

            if (0 == rang) {
                --message;
            }

            MPI_Send(&message, 1, MPI_INT, suivant, etiquette,
                    MPI_COMM_WORLD);

            if (0 == message) {
                break;
            }
        }
    }
}

```

3.2 Résultat

Avec un message initial fixé à 10, le compteur décroît bien de 9 à 0 au fil des dix tours complets de l'anneau, chaque processus affichant "Process X exiting" lorsqu'il reçoit la valeur 0, ce qui confirme la propagation correcte du message à travers tous les rangs.

4 Exercice 4 : Maître/Esclave

4.1 Principe

Le processus de rang 0 (le maître) lit une valeur entière sur l'entrée standard puis l'envoie individuellement, via une boucle de `MPI_Send`, à chacun des $n - 1$ autres processus (les esclaves), qui la réceptionnent chacun avec un `MPI_Recv` bloquant.

```

void maitre(int myrank, int nprocs) {
    int n, islave;

    scanf("%d", &n);

    for (islave = 1; islave < nprocs; islave++) {
        MPI_Send(&n, 1, MPI_INT, islave,
                ETIQUETTE_MASTERSLAVE, MPI_COMM_WORLD);
    }
}

void esclave(int myrank, int master) {

```

```

    int n;

    MPI_Recv(&n, 1, MPI_INT, master, ETIQUETTE_MASTERSLAVE,
             MPI_COMM_WORLD, MPI_STATUS_IGNORE);
}

```

4.2 Résultat

Les 15 esclaves confirment tous avoir reçu la valeur transmise par le maître ("The slave X done receiving 0 from the master 0"). Cette diffusion **séquentielle** par boucle de `MPI_Send` illustre une alternative naïve à `MPI_Bcast`, dont le coût croît linéairement avec le nombre de processus (contrairement à une diffusion en arbre).

5 Exercice 5 : Somme Parallèle

5.1 Principe et répartition de la charge

Cet exercice calcule la somme des éléments d'un grand tableau d'entiers de deux façons :

- **Séquentielle** : une simple boucle `for`, exécutée par un seul processus, servant de référence
- **Parallèle** : le tableau est réparti entre les p processus du communicateur, chacun calcule la somme de son segment, puis le maître (rang 0) collecte et additionne les sommes partielles

La répartition du tableau est équilibrée au mieux : si n n'est pas un multiple de p , les $n \bmod p$ premiers processus reçoivent un élément de plus que les autres.

```

void calculer_segment(int n, int nprocs, int rang, int *debut,
                     int *taille_segment) {
    int base = n / nprocs;
    int reste = n % nprocs;

    if (rang < reste) {
        *taille_segment = base + 1;
        *debut = rang * (base + 1);
    } else {
        *taille_segment = base;
        *debut = reste * (base + 1) + (rang - reste) *
            base;
    }
}

```

Le maître répartit les segments par des `MPI_Send` individuels vers chaque esclave, calcule la somme de son propre segment, puis collecte les sommes partielles par des `MPI_Recv` successifs :

```

long long somme_parallele(int tab[], int n, int rang, int nprocs,
                        MPI_Comm comm) {
    int debut, taille_segment;

```

```

    long long somme_locale = 0, somme_totale;

    calculer_segment(n, nprocs, rang, &debut, &taille_segment
    );

    if (0 == rang) {
        for (int r = 1; r < nprocs; r++) {
            int debut_r, taille_r;
            calculer_segment(n, nprocs, r, &debut_r,
                &taille_r);
            MPI_Send(&tab[debut_r], taille_r, MPI_INT
                , r, ETIQUETTE_SOMME, comm);
        }
        for (int i = debut; i < debut + taille_segment; i
            ++ ) somme_locale += tab[i];
    } else {
        int *segment_local = malloc(taille_segment *
            sizeof(int));
        MPI_Recv(segment_local, taille_segment, MPI_INT,
            0, ETIQUETTE_SOMME, comm, MPI_STATUS_IGNORE);
        for (int i = 0; i < taille_segment; i++)
            somme_locale += segment_local[i];
        free(segment_local);
    }

    somme_totale = somme_locale;

    if (0 == rang) {
        for (int r = 1; r < nprocs; r++) {
            long long somme_r;
            MPI_Recv(&somme_r, 1, MPI_LONG_LONG, r,
                ETIQUETTE_SOMME, comm,
                MPI_STATUS_IGNORE);
            somme_totale += somme_r;
        }
    } else {
        MPI_Send(&somme_locale, 1, MPI_LONG_LONG, 0,
            ETIQUETTE_SOMME, comm);
    }

    return (0 == rang) ? somme_totale : 0;
}

```


5.2 Complexité théorique

Opération	Complexité	Description
Somme séquentielle	$O(n)$	Une boucle sur les n éléments
Calcul local (par processus)	$O(n/p)$	Chaque processus somme son segment
Répartition + collecte	$O(p)$ messages	$2(p - 1)$ communications point-à-point (envoi des segments, retour des sommes partielles)
Temps parallèle (modèle simplifié)	$O(n/p + p)$	Calcul divisé par p , overhead de communication croissant avec p

TABLE 1 – Complexité de la somme parallèle

Ce modèle $O(n/p + p)$ explique directement le comportement attendu : pour n fixé, il existe un nombre de processus optimal au-delà duquel l’overhead de communication ($O(p)$) dépasse le gain apporté par la division du calcul ($O(n/p)$).

5.3 Protocole de mesure

Pour chaque taille de tableau $n \in \{10^4, 10^5, 5 \times 10^5, 10^6, 5 \times 10^6, 10^7\}$ et chaque nombre de processus $p \in \{1, 2, 4, 8, 16\}$, le temps séquentiel et le temps parallèle sont chacun mesurés **5 fois** avec `MPI_Wtime()` (encadré par des `MPI_Barrier` pour synchroniser le départ du chrono), et c’est la **médiane** de ces 5 essais qui est retenue. Ce choix, plutôt qu’une moyenne ou une mesure unique, limite l’influence d’un aléa ponctuel du système (ordonnanceur, interruption OS) sur le résultat final, sans lisser artificiellement une vraie tendance.

5.4 Résultats expérimentaux

n (taille)	p (processus)	Temps séq. (μs)	Temps par. (μs)	Accélération
10 000	1	64	52	1.22×
10 000	2	64	38	1.68×
10 000	4	64	49	1.30×
10 000	8	64	80	0.79×
10 000	16	64	68	0.94×
100 000	1	543	431	1.26×
100 000	2	543	273	1.99×
100 000	4	543	181	3.01×
100 000	8	543	245	2.22×
100 000	16	543	188	2.89×
500 000	1	1 975	1 660	1.19×
500 000	2	1 975	1 139	1.73×
500 000	4	1 975	706	2.80×
500 000	8	1 975	771	2.56×
500 000	16	1 975	622	3.17×
1 000 000	1	4 000	3 743	1.07×
1 000 000	2	4 000	2 655	1.51×
1 000 000	4	4 000	1 652	2.42×
1 000 000	8	4 000	1 226	3.26×
1 000 000	16	4 000	954	4.19×
5 000 000	1	20 404	16 936	1.20×
5 000 000	2	20 404	11 269	1.81×
5 000 000	4	20 404	7 779	2.62×
5 000 000	8	20 404	6 357	3.21×
5 000 000	16	20 404	5 232	3.90×
10 000 000	1	41 120	34 887	1.18×
10 000 000	2	41 120	23 835	1.73×
10 000 000	4	41 120	16 311	2.52×
10 000 000	8	41 120	13 546	3.04×
10 000 000	16	41 120	12 075	3.41×

TABLE 2 – Résultats de l'exercice 5 (médiane de 5 mesures)

5.5 Temps d'exécution en fonction de la taille du tableau

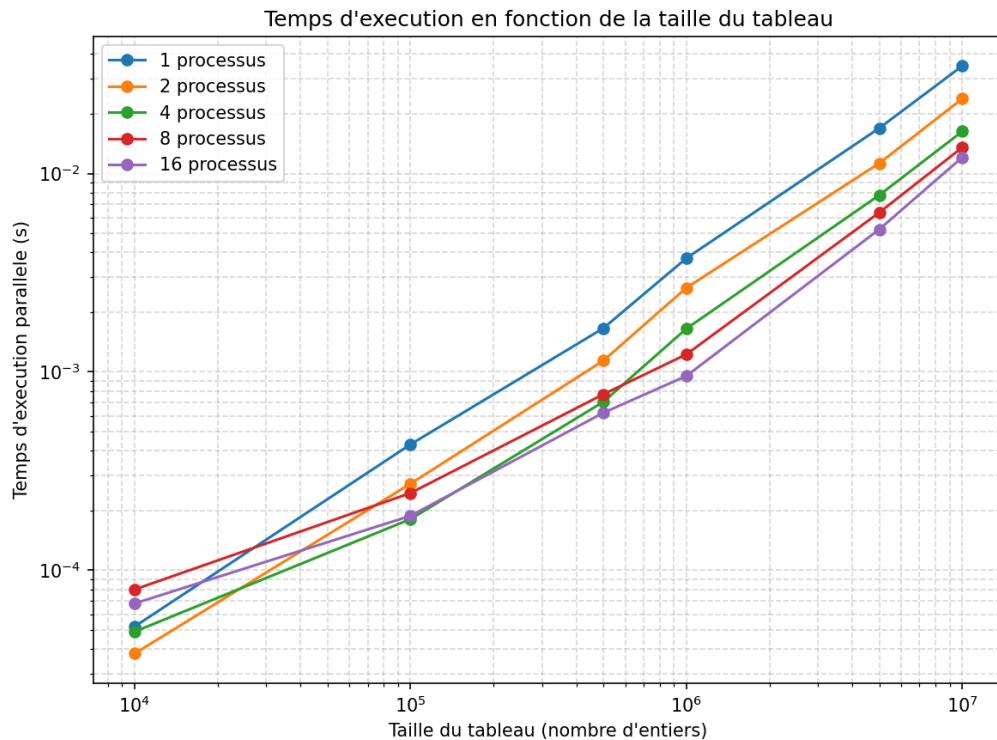


FIGURE 1 – Temps d'exécution parallèle en fonction de la taille du tableau (échelle log-log)

Observations :

- Toutes les courbes croissent quasi-linéairement en échelle log-log, ce qui est cohérent avec un temps de calcul dominé par $O(n)$ (réparti en $O(n/p)$) au-delà d'une certaine taille
- Pour les petites tailles ($n = 10\,000$), les courbes de 1, 2, 4, 8 et 16 processus sont proches, voire inversées par endroits : l'overhead de communication masque le gain de calcul
- Pour les grandes tailles ($n \geq 10^6$), l'écart entre les courbes s'élargit nettement : plus de processus procure un gain croissant, le calcul dominant de plus en plus l'overhead

5.6 Temps d'exécution en fonction du nombre de processus

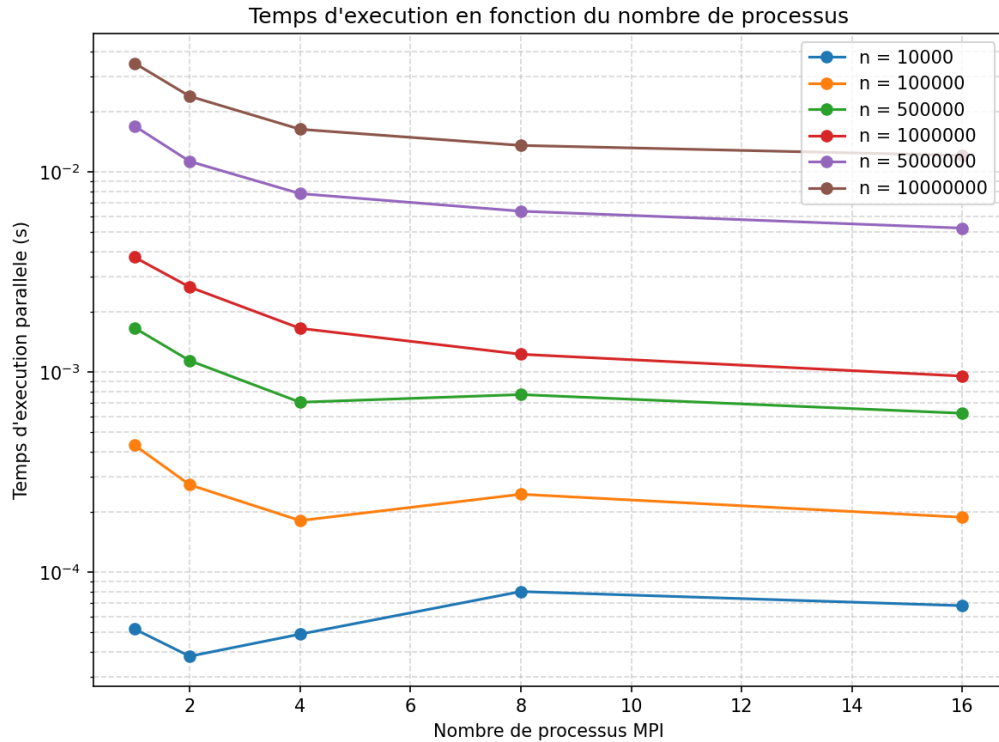


FIGURE 2 – Temps d'exécution parallèle en fonction du nombre de processus (échelle log en y)

Observations :

- Pour $n = 10\,000$ (courbe bleue), le temps ne diminue quasiment pas avec le nombre de processus, et remonte même légèrement à 8 processus : le tableau est trop petit pour que la parallélisation soit rentable
- Pour toutes les autres tailles, le temps décroît de façon monotone avec p , mais avec des **rendements décroissants** : le gain relatif entre 8 et 16 processus est nettement plus faible qu'entre 1 et 2 processus
- Cette décroissance en rendements décroissants est la signature classique d'un overhead de communication $O(p)$ qui grignote progressivement le gain de calcul $O(n/p)$

5.7 Accélération en fonction du nombre de processus

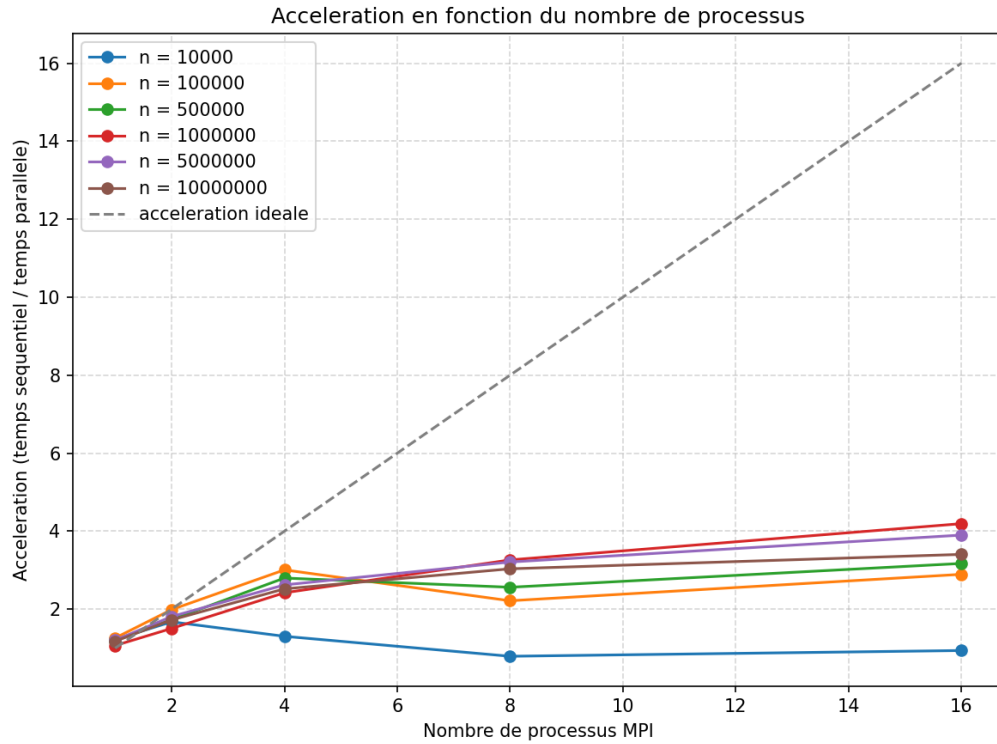


FIGURE 3 – Accélération (temps séquentiel / temps parallèle) en fonction du nombre de processus, comparée à l’accélération idéale

Observations :

- La droite pointillée grise représente l’accélération **idéale** (linéaire, $\text{acc} = p$) : aucune courbe ne s’en approche, ce qui est attendu pour une opération aussi légère par élément qu’une simple somme
- L’accélération maximale obtenue est d’environ **4.2×** (pour $n = 10^6$, $p = 16$), très loin de l’idéal $16\times$
- Pour $n = 10\,000$, l’accélération reste sous ou proche de $1\times$ pour tout p : la parallélisation est **contre-productive** sur un tableau aussi petit
- Plus n est grand, plus l’accélération progresse régulièrement avec p : c’est la loi d’Amdahl en action, la part de calcul « utile » devenant grande devant la part fixe de communication

5.8 Interprétation : la loi d’Amdahl à l’œuvre

Le modèle théorique $T_{\text{par}} \approx \frac{n}{p} \cdot t_{\text{calc}} + p \cdot t_{\text{comm}}$ (calcul divisé par p , overhead de communication croissant avec p) explique bien l’ensemble des observations :

- **Pour n petit** ($n = 10\,000$) : le terme $\frac{n}{p} \cdot t_{\text{calc}}$ devient négligeable dès $p = 2$ ou 4 , si bien que le temps parallèle est dominé par le terme fixe $p \cdot t_{\text{comm}}$, qui **croît** avec p . Ajouter des processus ne fait alors qu’ajouter de l’overhead : l’accélération stagne, voire régresse.

- **Pour n grand** ($n \geq 10^6$) : le terme $\frac{n}{p} \cdot t_{calc}$ reste dominant même pour $p = 16$, donc chaque doublement de p apporte encore un gain réel, bien que décroissant.
- L'opération sommée (une simple addition d'entiers) est **trop légère par élément** pour que le parallélisme MPI (qui a un coût de communication non négligeable par message) paie pleinement : c'est pourquoi même aux plus grandes tailles testées, l'accélération plafonne autour de $3.5\times$ – $4.2\times$ au lieu de s'approcher de $16\times$.

Ce résultat illustre un principe central du calcul parallèle par passage de messages : **le grain de calcul doit être suffisamment gros** par rapport au coût de communication pour que la parallélisation soit rentable — une leçon directement transposable à des algorithmes plus lourds (tri, produit matriciel, etc.) où le rapport calcul/communication est bien plus favorable.

Conclusion

Ce TP a permis de manipuler les briques de base de MPI (`MPI_Init`, `MPI_Comm_rank/size`, `MPI_Send/MPI_Recv`, `MPI_Barrier`, `MPI_Comm_split`, `MPI_Wtime`) à travers des schémas de communication variés : point-à-point simple, chronométré, en anneau, et en diffusion maître-esclave.

Résultats clés de l'exercice 5 :

- L'accélération obtenue reste **loin de l'idéal linéaire**, plafonnant à environ $4.2\times$ pour 16 processus sur les plus grands tableaux testés
- Pour les petits tableaux, la parallélisation est **contre-productive** : l'overhead de communication dépasse le gain de calcul
- Plus la taille du tableau augmente, plus l'accélération se rapproche (sans jamais l'atteindre) de l'idéal, conformément à la loi d'Amdahl
- La **médiane sur 5 mesures** par point s'est révélée nécessaire pour obtenir des courbes exploitables, un aléa de mesure ponctuel pouvant sinon fausser fortement un point isolé (accélération artificiellement basse ou haute sur une seule exécution)

Limite de l'expérience : l'opération sommée est arithmétiquement triviale ($O(1)$ par élément), ce qui maximise le poids relatif de la communication. Un TP futur avec une opération plus coûteuse par élément (tri, calcul matriciel, simulation) permettrait d'illustrer des accélérations bien plus proches de l'idéal, et de mieux visualiser le compromis calcul/communication au cœur du calcul parallèle par passage de messages.